Structured Pruning of Convolutional Networks: Local Hierarchical Clustering vs. Global Scoring under Accuracy Constraints

Prof. Russel Pears, Sai Naga Chaithanya Aavula, Tarun Sadarla
Department of Computer Science and Engineering, University of North Texas, Denton, Texas
Email: russel.pears@unt.edu, sainagachaithanyaaavula@my.unt.edu, tarunsadarla@my.unt.edu

***Abstract*—Structured pruning is a practical approach to compressing convolutional neural networks (CNNs) for deployment under tight latency and memory budgets [1], [2], [3], [4]. However, naive pruning often degrades accuracy and can disproportionately harm certain classes. We study two constrained pruning paradigms for vision models: (i) a *local* hierarchical clustering (Local HC) approach that prunes similar filters within each layer, and (ii) a *global* scoring (Global) approach that ranks filters using an ILR-style multi-signal score built from activation RMS, HRank, and batch-normalization gamma (BN-γ) statistics, combined with a compute-aware benefit metric; Taylor-style gradient saliency is retained for ablations. In both cases, the user specifies maximum allowable drops in overall and per-class accuracy, and the pruning algorithm enforces these guard rails on-line via step-wise and cumulative checks.**

We evaluate both paradigms on two architectures (a custom 5-layer CNN with batch normalization and VGG16-BN (hereafter VGG16)) and two binary classification tasks derived from CelebA and CIFAR-10 (hereafter CIFAR10). On CelebA, Global pruning achieves up to 70 % parameter reduction and a 1.28 × latency speedup on the 5-layer CNN while keeping overall accuracy within 2 percentage points and per-class drops within 6 percentage points of the baseline. Local HC is more conservative, yielding 30 % to 46 % parameter reduction and 1.03 × to 1.04 × speedup under the same guard rails, and sometimes slightly improving the accuracy of the harder class. On CIFAR10, where we relax the guard rails due to the smaller evaluation set, Global pruning still stays within the relaxed limits and reaches up to 2.05 × speedup. We also quantify guard-rail budget usage and the amortized return on investment (ROI) of pruning overhead, and show a clear trade-off: Global scoring with floors, guard rails, and ILR-style multi-signal scoring delivers higher compression and compute savings at the cost of greater algorithmic and engineering complexity, while Local HC pruning is simpler and safer but leaves substantial redundancy in the network.

I. Introduction

Deep convolutional networks are widely deployed in latency- and memory-constrained settings such as mobile devices and edge accelerators. Structured pruning—removing channels, filters, or entire blocks—is a standard method to reduce computational cost while preserving accuracy. In practice, however, aggressive pruning can produce unpredictable accuracy degradation and class-specific failures, which are unacceptable in many real-world applications.

We therefore study how to prune convolutional networks *under explicit accuracy guard rails*. A practitioner specifies tolerances on overall and per-class accuracy as allowable drops in percentage points relative to a pre-trained baseline. The pruning algorithm must respect these constraints on-line: each proposed structural change (e.g., removing a group of filters) is only committed if the pruned model remains within the specified error budget.

We compare two complementary pruning paradigms:

- **Local HC pruning.** We employ a layer-local clustering design that clusters filters within each layer based on similarity and iteratively merges or removes redundant filters. Decisions are local to a layer or block and never directly compare filters across distant layers. Guard rails are enforced by testing candidate merges and rolling back changes that violate constraints.
- **Global scoring pruning.** Building on prior work on Taylor-based channel saliency, HRank feature-map ranking, BN-γ-based channel slimming, and structured pruning under resource constraints [3], [5], [6], [7], [8], [4], we compute global importance scores by fusing these signals into a single ranking and then prune filters from that Global ranking. The Global shortlist is further passed through a compute-aware *benefit* metric that accounts for expected reductions in multiply-accumulates (MACs) and parameters, subject to hard floors on surviving channels per block. Guard rails are enforced both per-chunk and cumulatively along the pruning trajectory, with rollback when tests fail.

Both approaches are implemented in a unified PyTorch framework with shared infrastructure: caching of convolutional shapes and scores, explicit handling of dynamic `ChannelGate` modules, safe rollback of model structure and state, and careful control over evaluation functions (including optional graph-compilation passes provided by the deep learning framework once the model structure is fixed).

We experiment with two vision architectures and two binary classification tasks. The first architecture is a custom 5-layer CNN with batch normalization (5-CNN) designed for CelebA face attribute classification, which serves as a compact baseline for comparison with VGG16. The second architecture is the standard VGG16 network with batch normalization [9], [10], adapted to binary classification by replacing the final classifier. The first experiment is based on a binary male/female classification task constructed from CelebA [11], with a large evaluation set of roughly 19k images and natural class

imbalance. The second experiment also involves a binary truck-vs-ship classification task derived from CIFAR10 [12] by restricting to classes 9 (truck) and 8 (ship), with a smaller evaluation set of roughly 2k images. Because evaluation set size affects the stability of accuracy estimates, we apply stricter guard rails on CelebA (± 2 percentage points overall, ± 6 per class) and relaxed guard rails on CIFAR10 (± 3 overall, ± 9 per class).

Our main contributions are:

- A comparison of two structured pruning paradigms—Local HC and Global scoring with compute-aware selection—within a common guard-rail framework that constrains overall and per-class accuracy [4];
- A detailed engineering design that makes Global pruning robust over long runs (floors, soft locks, cache management, shape probing, rollback) together with working code for both approaches;
- Six canonical experiments: Local vs Global on two architectures for CelebA, plus Global-only runs on both architectures for CIFAR10, showing that Global pruning achieves substantially higher parameter and MAC reductions at similar overall accuracy cost, while Local HC pruning is simpler and more conservative, sometimes improving the harder class;
- Derived metrics for guard-rail budget usage and economic ROI that connect pruning to deployment constraints;
- A quantitative and qualitative comparison of when the extra complexity of Global pruning is justified and when Local HC remains an attractive alternative.

The rest of the paper is organized as follows. Section II reviews related work on structured pruning and constrained compression. Section III describes the datasets, architectures, and guard rails. Section IV presents the pruning algorithms in pseudo-code form. Section V details the experimental setup. Section VI reports empirical results, Section VII discusses insights and trade-offs, Section VIII outlines limitations and future work, and Section IX concludes.

II. Related Work

Early work on network pruning focused on *unstructured* weight pruning, in which individual scalar weights are removed based on their absolute value (magnitude) [13], [14]. This produces sparse weight matrices with irregular sparsity patterns. However, such unstructured sparsity typically yields only marginal end-to-end speedups on commodity GPUs and CPUs (often well below $1.2\times$ unless sparsity is extremely high) because standard dense linear algebra libraries cannot fully exploit the irregular structure [4]. Structured pruning methods instead remove entire channels, filters, or blocks, which directly reduce multiply-accumulates (MACs) and can be exploited by standard dense kernels [1], [2], [4].

Taylor-based pruning estimates the importance of channels or filters using first-order approximations of the loss with respect to activations or weights [3]. HRank-style methods evaluate channel importance via the rank or energy of feature maps [5], while BN-γ pruning uses the scale parameters of batch normalization layers as a surrogate for channel importance [6], [7]. Several recent works combine multiple saliency signals and introduce constraints on layer-wise sparsity or resource budgets [8], [4].

Hierarchical clustering and other similarity-based methods prune by grouping filters within a layer and merging or discarding redundant clusters. These methods naturally respect local structure but may miss global trade-offs across layers. In contrast, global ranking methods score filters across the whole network and rely on constraints (e.g., per-layer floors) to avoid catastrophic degradation. Our work lies at this intersection: we explicitly compare a layer-local HC method to a Global scoring method, both under explicit accuracy guard rails and with strong engineering emphasis.

III. Methodology

A. Overview

At a high level, our methodology has four stages. First, we train baseline models for each combination of architecture (5-CNN, VGG16) and dataset (CelebA, CIFAR10) using a fixed training protocol. Second, we define accuracy guard rails and hard per-block channel floors that any pruning trajectory must respect. Third, starting from each baseline, we apply either Local HC pruning or Global scoring and benefit-aware pruning, treating pruning as a sequence of small structural edits interleaved with evaluation under the guard rails. Finally, we measure accuracy, compression, MACs, latency, guard-rail budget usage, and economic ROI for each run, and compare the resulting six canonical experiments.

The subsections below describe the architectures and floors, datasets and guard rails, the guard-rail mechanism itself, notation, and then the two pruning algorithms in more detail.

B. Architectures and Hard Floors

We consider two convolutional architectures.

5-layer CNN with batch normalization. The first architecture is a custom 5-layer CNN with batch normalization after each convolution.

VGG16. The second architecture is VGG16 with batch normalization [9], [10], adapted to binary classification by replacing the final classifier.

For both architectures we define *hard floors* on surviving channels per block. A hard floor for block ℓ is an integer F_ℓ that specifies the minimum number of channels allowed to remain in that block after pruning; once a block reaches its floor, no further pruning is permitted in that block. These floors control pruning aggressiveness and act as safety constraints independent of accuracy.

In our canonical experiments we use a single floor scheme, which we denote *S2*, for both the Local HC and Global approaches. The label S2 is a shorthand for the second candidate floor configuration from a sweep over several schemes (S0, S1, S2, ...); it does *not* mean that each block is reduced to two channels. Instead, S2 assigns different floor values F_ℓ across blocks (e.g., higher floors in early, compute-intensive blocks (those with large MAC budgets) and lower floors in

later blocks) to achieve a moderately aggressive but still safe level of pruning.

C. Datasets and Accuracy Guard Rails

We study two binary classification tasks derived from standard vision datasets.

CelebA (male/female). We use CelebA [11] to predict the *male* attribute. The data pipeline builds three disjoint splits: a training set of 10,000 images, a prune reference set of 6,000 images that is used during pruning for batch-normalization recalibration and auxiliary logging, and a large evaluation set of 19,867 images with natural class proportions (approximately 57% vs. 43%). All images are resized to 224×224 and normalized with ImageNet statistics. For the Global method, the main scoring signals are computed on a class-aware "active" subset of the evaluation cache derived from this evaluation split (Section III), so that scoring and guard-rail checks share the same distribution.

Given this large evaluation set, we adopt *strict* guard rails for CelebA:

- Overall accuracy drop ≤ 2 percentage points (pp) relative to the baseline.
- Per-class accuracy drop ≤ 6 pp for each class.

CIFAR10 (binary). To probe a different regime, we derive a binary task from CIFAR10 [12] by restricting to two CIFAR10 classes and remapping them to a binary label. In our canonical experiments we fix this pair to classes 9 (truck) and 8 (ship), configured via `CIFAR_CLASSES` = (9, 8) in the scripts so that the same truck-vs-ship task is used across all runs. We again form three splits, but the evaluation set is smaller (around 2,000 images) and class-balanced by construction. The training and prune reference splits use similar sizes to CelebA (10k and 6k), and images are resized to 224×224 to match the 5-CNN and VGG16 architectures.

Because the evaluation set is smaller and accuracy estimates are noisier, we use *relaxed* guard rails for CIFAR10:

- Overall accuracy drop ≤ 3 pp.
- Per-class accuracy drop ≤ 9 pp.

These limits define the error budgets that the pruning algorithms must respect throughout their trajectories.

D. Guard Rail Mechanism

Both pruning approaches share the same conceptual guard-rail mechanism. Let A_{base} be baseline overall accuracy and $A_{c,\text{base}}$ baseline accuracy for class $c \in \{0, 1\}$. After each proposed structural change (e.g., pruning a chunk of filters), we run an evaluation on a held-out subset and compute new accuracies A and A_c.

For each dataset we define maximum allowed drops $\Delta^{\max}_{\text{overall}}$ and $\Delta^{\max}_{\text{class}}$ and enforce

$$A_{\text{base}} - A \leq \Delta^{\max}_{\text{overall}}, \tag{1}$$

$$A_{c,\text{base}} - A_c \leq \Delta^{\max}_{\text{class}} \quad \text{for } c \in \{0, 1\}. \tag{2}$$

In the Global approach we further distinguish per-step and cumulative drops, maintaining running estimates of cumulative degradation. In addition to the dataset-level budgets above, we enforce per-chunk limits on how much overall and per-class accuracy are allowed to change in a single pruning step (on CelebA, for example, at most 1 pp overall and 3 pp per class, with cumulative drops capped at the 2 pp and 6 pp budgets). When a proposed change exceeds any per-step or cumulative bound, we roll back the change and temporarily lock the corresponding filters or layers. In the Local HC method guard rails are applied at the level of candidate merges within a block, with cumulative tracking but fewer explicit budget parameters. Both methods use the same evaluation pipeline, ensuring consistent guard-rail enforcement.

E. Notation and Shorthands

For brevity, we use the following shorthands:

- **5-CNN**: custom 5-layer CNN with batch normalization,
- **VGG16**: VGG16-BN architecture,
- **Local HC**: local hierarchical clustering pruning approach,
- **Global**: Global scoring pruning approach,
- **CIFAR10**: binary task derived from CIFAR10.

F. Local Hierarchical Clustering Pruning

The Local HC approach operates independently within each convolutional block. For a given block, we build a feature matrix $X_\ell \in \mathbb{R}^{N \times C^{(\ell)}_{\text{out}}}$ by attaching a hook to the block's spatial pooling operation and running the model on a subset of the evaluation set. Each row of X_ℓ corresponds to one image, and column i contains the global-average-pooled activation of output channel i across those N examples. We then ℓ_2-normalize each column to obtain vectors $v_{\ell,i} \in \mathbb{R}^N$, and define the distance between channels i and j in layer ℓ as

$$d_\ell(i, j) = 1 - \frac{v_{\ell,i}^\top v_{\ell,j}}{\|v_{\ell,i}\|_2 \, \|v_{\ell,j}\|_2},$$

that is, one minus the cosine similarity of their normalized activation vectors (optionally after centering for a Pearson-style variant). The distance between two clusters C_a and C_b is the average pairwise distance

$$D_\ell(C_a, C_b) = \frac{1}{|C_a| \, |C_b|} \sum_{i \in C_a} \sum_{j \in C_b} d_\ell(i, j).$$

Hierarchical clustering proceeds by repeatedly merging the closest pair of clusters (with smallest $D_\ell(C_a, C_b)$) while the total number of live filters in the layer remains above the hard floor F_ℓ. When clusters C_a and C_b are merged, we keep a representative filter whose weights are the element-wise average of the filters in $C_a \cup C_b$, and we mark the other filters in the merged cluster as candidates for removal.

For each accepted removal of an output channel in layer ℓ, we also compute a simple benefit metric that captures the amount of computation and parameters eliminated in that layer. Let H_ℓ and W_ℓ be the spatial dimensions of the convolutional output feature map (before pooling) and k_ℓ the kernel size. The MAC cost associated with a single output channel is

$$\text{MAC}^{\text{per-ch}}_\ell = H_\ell W_\ell \, C^{(\ell)}_{\text{in}} \, k^2_\ell,$$

and the parameter cost is

$$\text{Param}_\ell^{\text{per-ch}} = C_{\text{in}}^{(\ell)}\, k_\ell^2.$$

If a merge results in m channels being removed in layer ℓ, we record its benefit as

$$B_\ell = m \cdot \left(\lambda_{\text{MAC}}\, \text{MAC}_\ell^{\text{per-ch}} + \lambda_{\text{Param}}\, \text{Param}_\ell^{\text{per-ch}}\right),$$

with non-negative weights λ_{MAC} and λ_{Param} (in our experiments we use a MAC-dominated weighting). This benefit is recorded for later analysis and reporting (for example, per-layer MAC breakdowns); the actual merge decisions are driven by similarity, accuracy-based guard rails, and intrinsic consistency checks rather than by this compute metric directly.

At each candidate merge or removal, we modify the network structure (updating subsequent layers or an intermediate `ChannelGate` module that masks pruned channels) and evaluate the pruned model on the evaluation set. If the guard rails on overall and per-class accuracy are violated, we roll back the change and mark the pair or cluster as ineligible for further merges. This process continues until no safe merges (i.e., merges that both respect the accuracy guard rails and keep each layer above its floor) remain or all layers have reached their floors.

Because all decisions are local to a block or layer, the method tends to produce smooth, layer-wise sparsity patterns and naturally respects the architectural structure. However, it does not explicitly compare the relative importance of filters across blocks and can therefore under-utilize the Global accuracy budget.

G. Global Scoring and Benefit-Aware Pruning

The Global approach maintains a single pool of candidate filters across all prunable layers. For each filter we compute several importance signals inspired by prior pruning methods [3], [5], [6], [7]. In the implementation, these signals are evaluated on a class-aware "active" subset of the evaluation cache, built from the evaluation loader, so that scoring and guard-rail checks are tightly coupled. In all canonical experiments we cap this active subset at a few thousand images and construct it by balanced, hardness-aware sampling from the evaluation split: we enforce a per-class floor on sample counts, prioritize ambiguous or high-entropy examples, and reuse the resulting subset for both scoring and guard checks.

Let $z_{\ell,i}(x) \in \mathbb{R}^{H_\ell \times W_\ell}$ denote the post-activation feature map of channel i in layer ℓ for input x, and let $\mathcal{D}_{\text{score}}$ denote the class-aware active subset of the evaluation cache $(X_{\text{active}}, y_{\text{active}})$ used for scoring and guard checks.

a) Taylor saliency.: Following Molchanov et al. [3], we approximate the change in loss if channel (ℓ, i) is zeroed out using a first-order Taylor expansion. Let $L(x, y)$ be the loss for example (x, y) and $\frac{\partial L}{\partial z_{\ell,i}(x)}$ be the gradient with respect to the feature map. The Taylor saliency score is

$$T_{\ell,i} = \mathbb{E}_{(x,y)\in\mathcal{D}_{\text{score}}}\left[\left\|\frac{\partial L}{\partial z_{\ell,i}(x)} \odot z_{\ell,i}(x)\right\|_1\right],$$

where $\odot$ denotes element-wise multiplication and $\|\cdot\|_1$ is the ℓ_1 norm over spatial positions. Channels with smaller $T_{\ell,i}$ are considered less important.

b) HRank-style feature-map energy.: Motivated by HRank [5], which uses feature-map rank to measure importance, we compute an energy-style proxy based on the Frobenius norm of the feature map:

$$H_{\ell,i} = \mathbb{E}_{(x,y)\in\mathcal{D}_{\text{score}}}\left[\|z_{\ell,i}(x)\|_F\right].$$

Channels with smaller $H_{\ell,i}$ carry less activation energy and are candidates for pruning.

c) BN-γ score.: For layers followed by batch normalization, we use the absolute value of the BN scale parameter $\gamma_{\ell,i}$ as a pruning signal [6], [7]:

$$G_{\ell,i} = |\gamma_{\ell,i}|.$$

Channels with smaller $G_{\ell,i}$ are considered less important.

d) Activation RMS (ActRMS).: As an additional forward-only signal, we measure the root-mean-square (RMS) activation of each channel. For channel (ℓ, i) we define

$$A_{\ell,i} = \mathbb{E}_{(x,y)\in\mathcal{D}_{\text{score}}}\left[\text{RMS}\big(z_{\ell,i}(x)\big)\right],$$

where RMS denotes the root-mean-square over the batch and spatial dimensions. Channels with smaller $A_{\ell,i}$ tend to be weakly activated across the scoring subset.

e) Global normalization, fusion, and inside-layer ranking (ILR).: In the implemented Global method, each raw signal is normalized *globally* across all layers rather than layer by layer. Let

$$T_{\min} = \min_{\ell,i} T_{\ell,i}, \quad T_{\max} = \max_{\ell,i} T_{\ell,i},$$

and define

$$\hat{T}_{\ell,i} = \frac{T_{\ell,i} - T_{\min}}{T_{\max} - T_{\min} + \varepsilon},$$

with a small ε to avoid division by zero. We normalize $H_{\ell,i}$ and $G_{\ell,i}$ analogously to obtain $\hat{H}_{\ell,i}$ and $\hat{G}_{\ell,i}$; this makes channels from different layers directly comparable on each signal.

We then form a fused importance score

$$S_{\ell,i} = \alpha_T \hat{T}_{\ell,i} + \alpha_H \hat{H}_{\ell,i} + \alpha_G \hat{G}_{\ell,i},$$

where $\alpha_T, \alpha_H, \alpha_G \geq 0$ and $\alpha_T + \alpha_H + \alpha_G = 1$. When we run the Global scorer *without* ILR (ablation mode), we use a slightly Taylor-dominated weighting, $\alpha_T = 0.5$, $\alpha_H = 0.3$, and $\alpha_G = 0.2$, with configuration toggles that allow alternative choices. In the canonical experiments reported in this paper, however, we enable an inside-layer ranking module and use it as the primary importance score; the fused Taylor/HRank/BN-γ score is retained for ablations and auxiliary logging.

In addition to this fused Global score, we compute a separate *inside-layer ranking (ILR)* score. The ILR module builds a per-layer score by combining several layer-local signals (running activation RMS, BN-γ magnitude, and HRank-style energy) and then normalizing within each layer. Concretely, for each layer ℓ we form per-channel signals $A_{\ell,i}$, $G_{\ell,i}$, and $H_{\ell,i}$ as

defined above and apply a per-layer min–max normalization to obtain

$$\tilde{A}_{\ell,i}, \quad \tilde{G}_{\ell,i}, \quad \tilde{H}_{\ell,i} \in [0,1].$$

We then define the ILR score as

$$I_{\ell,i} = \frac{w_A \tilde{A}_{\ell,i} + w_G \tilde{G}_{\ell,i} + w_H \tilde{H}_{\ell,i}}{w_A + w_G + w_H},$$

with non-negative weights w_A, w_G, w_H (in our implementation we use $w_A = 0.6$, $w_G = 0.4$, and $w_H = 0.2$). Thus $I_{\ell,i} \in [0,1]$ and smaller values correspond to less important channels within layer ℓ. ILR is computed entirely from forward statistics (activations, BN-γ, and feature-map energy) and does not use Taylor gradients.

In our implementation ILR is computed by a distinct scoring routine, and a configuration flag controls how it interacts with S. In the default setting used for all canonical runs (`ILR_ENABLE = True`, `ILR_REPLACE_GLOBAL = True`), the ILR score *replaces* the fused Taylor/HRank/BN-γ score when constructing the shortlist, so the final Global importance score $S^{\text{final}}_{\ell,i}$ coincides with $I_{\ell,i}$. An alternative mode (`ILR_REPLACE_GLOBAL = False`) fuses ILR as an additional component in the weighted sum for ablation studies. In both modes ILR only changes how we rank channels; the guard-rail checks and compute-aware benefit calculations that follow are unchanged.

Empirically, we found that relying on ILR as the primary Global score yields a better trade-off between cost, stability, and reproducibility than using Taylor gradients in the inner loop. ILR is forward-only and inexpensive to evaluate across many pruning rounds, and it is less sensitive to training-mode or compiler details (e.g., mixed precision, `torch.compile`, or subtle changes in loss gradients) than gradient-based saliency. For this reason we fix ILR as the canonical Global scoring mechanism in all reported experiments, while treating Taylor-based variants as ablations and avenues for future work.

At each Global pruning round we perform the following stages:

1) **Score stage.** Using the scoring dataset $\mathcal{D}_{\text{score}}$ (the class-aware active subset of the evaluation cache), we compute $T_{\ell,i}$, $H_{\ell,i}$, and $G_{\ell,i}$ for all active filters, normalize them globally to obtain $\hat{T}_{\ell,i}$, $\hat{H}_{\ell,i}$, and $\hat{G}_{\ell,i}$, form fused scores $S_{\ell,i}$ with the configured weights $(\alpha_T, \alpha_H, \alpha_G)$, and compute the ILR scores $I_{\ell,i}$ from activation RMS, BN-γ, and HRank signals. A configuration flag then determines the final Global importance score: in our canonical runs we set `ILR_ENABLE = True` and `ILR_REPLACE_GLOBAL = True` so that $S^{\text{final}}_{\ell,i} = I_{\ell,i}$, whereas in ablation mode we either use $S_{\ell,i}$ alone or fuse $I_{\ell,i}$ into $S_{\ell,i}$. Channels with smaller $S^{\text{final}}_{\ell,i}$ are less important.
2) **Shortlist stage.** Among filters that are not locked and whose layers are above their hard floors, we build a shortlist consisting of the lowest-scoring fraction (e.g., the bottom 25% in terms of $S^{\text{final}}_{\ell,i}$). This shortlist contains the channels that are most likely to be pruned in the current round.
3) **Benefit stage.** For each shortlisted filter we estimate a compute-aware *benefit* that captures expected reductions in convolutional MACs if that filter is removed. Let

$$b_{\text{this}}(\ell, i)$$

denote the MACs saved in the current convolutional layer, computed from cached convolutional output shapes (H_ℓ, W_ℓ) and kernel sizes; let

$$b_{\text{next}}(\ell, i)$$

denote the MACs saved in the next convolutional layer (if any) because its input channel count shrinks when channel (ℓ, i) is pruned; and let

$$b_{\text{lin}}(\ell, i)$$

denote the MACs saved in the first fully connected (Linear1) layer when pruning a channel in the last convolutional block. We combine these into a single MAC-only benefit

$$B_{\ell,i} = b_{\text{this}}(\ell, i) + \alpha_{\text{next}}\, b_{\text{next}}(\ell, i) + \beta_{\text{lin}}\, b_{\text{lin}}(\ell, i),$$

where α_{next} and β_{lin} are non-negative coefficients set via configuration. Parameter savings are implicitly captured through these MAC reductions, especially in compute-intensive layers. For a fixed layer ℓ this benefit is constant across channels i, because removing any one output channel from that layer saves the same amount of computation; $B_{\ell,i}$ therefore varies across layers but not within a layer.

Selection and chunking. Given the Global budget for the current round (specified as a maximum number of channels that may be removed in that round), we select filters using a lexicographic ordering on score and benefit rather than a single scalar $R_{\ell,i}$. Concretely, we first collect all eligible filters (those that are not soft-locked and whose layers are above their floors) and sort them by increasing $S^{\text{final}}_{\ell,i}$; ties are broken by *decreasing* compute-aware benefit $B_{\ell,i}$. This produces a global order in which low-score, high-benefit filters appear first. We then traverse this ordering and greedily take filters while respecting the per-round budget and per-layer floors. Finally, we partition the selected set into small groups ("chunks") of size 2–4 filters; each chunk is simply a small subset of channels that will be proposed for removal together in a single guarded update.
4) **Pruning under guard rails.** For each chunk in turn, we:
   - Save a lightweight snapshot of the current model state (including weights and channel counts).
   - Apply structural changes corresponding to the chunk: prune the selected channels, update `ChannelGate` masks, and adjust downstream weights and shapes.
   - Evaluate the modified model on the held-out evaluation subset and compute new overall and per-class accuracies.

- If any of the step-wise or cumulative guard-rail constraints on overall or per-class accuracy are violated, we discard this proposal by restoring the saved snapshot and *soft-lock* the affected channels or layers, meaning they are temporarily excluded from future shortlists. Otherwise, we keep the change, update the running accuracies and guard-rail budgets, and record the accepted chunk in the pruning trajectory.

5) **Soft locks and floors.** A *floor* for block ℓ is the hard lower bound F_ℓ on surviving channels; once the number of active channels in ℓ equals F_ℓ, that block is permanently excluded from further pruning. A *soft lock* is a temporary exclusion of a channel or layer from consideration, triggered when repeated proposals involving that channel or layer violate guard rails. Soft-locked channels and layers may be reconsidered in later rounds if budgets allow, but in practice they steer the algorithm away from unstable regions of the architecture.

This procedure repeats for a fixed number of rounds (e.g., 75) or until no safe pruning opportunities remain under the accuracy guard rails and hard floors. To make it more efficient, we cache convolutional output shapes and reuse score computations whenever possible, invalidating caches only when structural changes affect shapes. We also carefully manage the evaluation functions: during phases where the architecture is stable for a given experiment (for example, when repeatedly evaluating the same pruned model to measure latency), we optionally apply a graph-compilation pass provided by the deep learning framework, which traces the evaluation graph and generates optimized fused kernels. This compilation step reduces Python overhead and improves throughput, but we avoid it during phases where the model structure is still changing frequently.

IV. Algorithms

To make the pruning procedures more concrete, we now present explicit pseudo-code for both approaches. These algorithms implement the methodology described in Section III and are the exact procedures used in all experiments. We emphasize high-level control flow and guard-rail enforcement; implementation details such as `ChannelGate` manipulation, shape caching, and rollback snapshots follow directly from the codebase.

A. Local Hierarchical Clustering Pruning

Algorithm 1 describes the Local HC pruning procedure, which operates independently within each convolutional layer while enforcing accuracy guard rails and hard per-block floors.

Algorithm 1 Local HC pruning under guard rails

Input: pretrained model M, evaluation loader L_{eval}, configuration T (including layer order, floors, and guard parameters), device d.
Output: pruned model M' that respects guard rails and hard floors.

1) Insert `ChannelGate` modules after each prunable convolutional layer indicated in T.`ALT_ORDER` to enable channel-wise masking.
2) Build a forward-evaluation wrapper `FWD_EVAL` (optionally compiled) that runs M in evaluation mode on device d.
3) Run a baseline evaluation on L_{eval} to obtain overall accuracy A_{base} and per-class accuracies $A_{c,\text{base}}$ for $c \in \{0, 1\}$. Initialize a trajectory log with this baseline.
4) Build an evaluation cache $(X_{\text{eval}}, y_{\text{eval}})$ by materializing a subset of the evaluation set on device d, and optionally construct:
   - a BN calibration cache X_{bn} for batch-normalization refresh,
   - an "active" subset of important examples via `pick_active_indices_stratified`.
5) For each prunable layer ℓ in T.`ALT_ORDER`:
   a) Extract activation-based filter representations by building the feature matrix X_ℓ from pooled activations and forming normalized vectors $v_{\ell,i}$ as in Section III-F, and build an `HCProximity` structure over filters in ℓ, which maintains:
      - a set of live filter indices,
      - a similarity heap over filter pairs,
      - cluster membership for merged filters.
6) Initialize state variables: last accepted accuracy $A_{\text{last}} \leftarrow A_{\text{base}}$, per-class accuracies $A_{c,\text{last}} \leftarrow A_{c,\text{base}}$, rollback counters, and safety-brake statistics.
7) **Loop over candidate merges** while there exist layers and pairs that can be merged without violating hard floors:
   a) Select a layer ℓ and pop the most similar filter pair (i, j) from its `HCProximity` heap.
   b) **Propose a merge:**
      - Combine filters i and j according to the chosen linkage (e.g., average weights), or drop the less important filter and update the surviving filter.
      - Update `ChannelGate` masks, cluster memberships, and any downstream connectivity affected by the removal.
   c) Optionally perform a light BN recalibration on X_{bn} every `BN_REFRESH_EVERY_MERGES` accepted merges or per-layer mini-batches.
   d) Evaluate the modified model on a subset of $(X_{\text{eval}}, y_{\text{eval}})$ (or the active subset) using `FWD_EVAL` to obtain new accuracies A and A_c.
   e) **Guard rail check:**
      - Compute overall drop $\Delta A = A_{\text{base}} - A$ and per-class drops $\Delta A_c = A_{c,\text{base}} - A_c$.
      - If ΔA or any ΔA_c exceeds the Local thresholds (e.g., `ROLLBACK_ACC_DROP`, `PERCLASS_DRIFT_THRESH`, or the safety-brake thresholds), then:
        i) *Reject* the merge and rollback structural changes, optionally undoing several recent merges using incremental rollback.
        ii) Mark the pair (and possibly the layer) as ineligible for further merges if repeated violations occur.
      - Otherwise:
        i) *Accept* the merge, update $A_{\text{last}} \leftarrow A$, $A_{c,\text{last}} \leftarrow A_c$, and append a new entry to the trajectory log.
        ii) Update per-layer statistics (channels, MACs) and the `HCProximity` structure for ℓ.
   f) Periodically check a safety brake: if the cumulative accuracy drop or the number of merges since the last checkpoint exceeds configured limits, stop merging and move to the next layer or terminate.
8) After no more safe merges are available or floors are reached in all layers, perform a final BN recalibration (if enabled) and run a full evaluation on L_{eval} to obtain final metrics.
9) Assemble and save a summary containing baseline and final metrics, per-layer channel counts, and trajectory statistics.

B. Global Scoring and Benefit-Aware Pruning

Algorithm 2 gives the Global scoring and benefit-aware pruning procedure, which maintains a single pool of candidate filters across layers and prunes them under the same guard-rail and floor constraints.

Algorithm 2 Global scoring and benefit-aware pruning under guard rails

Input: pretrained model M, evaluation loader L_{eval}, prune reference loader L_{prune} (used primarily for batch-normalization calibration and auxiliary logging), configuration T (layer order, floors, scoring weights, guard parameters), device d.
Output: pruned model M' and pruning trajectory satisfying guard rails and hard floors.

1) Insert `ChannelGate` modules after each prunable convolution and build `layer_specs` (conv, BN, pool paths) from T.`ALT_ORDER`.
2) Initialize a `ChannelTracker` with channel counts and hard floors, and a `PruneReporter` for round-by-round logging.
3) Build an evaluation wrapper `FWD_EVAL` for M (optionally compiled), run a baseline evaluation on L_{eval} to obtain A_{base}, $A_{c,\text{base}}$, baseline latency, and per-layer MAC/parameter statistics, and materialize an evaluation cache $(X_{\text{eval}}, y_{\text{eval}})$ with:
   - an active subset $(X_{\text{active}}, y_{\text{active}})$ for fast guard checks and scoring;
   - a BN calibration cache $(X_{\text{bn}}, y_{\text{bn}})$ (which may draw on L_{prune} and/or L_{eval}).
4) Initialize Global state: last good model state, last accepted accuracies, cumulative guard budgets, chunk size and round pace, and soft-lock structures for layers/filters.
5) **For each Global pruning round** $r = 1, \ldots, R_{\max}$:
   a) If all layers are at their floors or guard budgets (cumulative overall and per-class drops) are exhausted, terminate.
   b) **Score stage:** using the class-aware active subset $(X_{\text{active}}, y_{\text{active}})$, compute per-filter Taylor, HRank, and BN-γ scores; normalize each signal globally across all layers and fuse them with the configured weights into a single importance score $S_{\ell,i}$ per filter. In addition, compute ILR scores $I_{\ell,i}$ from activation RMS, BN-γ, and HRank signals as described in Section III. A configuration flag determines the final Global importance score: in our canonical runs we set `ILR_ENABLE` $=$ `True` and `ILR_REPLACE_GLOBAL` $=$ `True` so that $S^{\text{final}}_{\ell,i} = I_{\ell,i}$, whereas in ablation mode we either use $S_{\ell,i}$ alone or fuse $I_{\ell,i}$ into $S_{\ell,i}$. Channels with smaller $S^{\text{final}}_{\ell,i}$ are less important.
   c) **Shortlist stage:** among filters that are not soft-locked and whose layers are above floors, form a shortlist consisting of the lowest-scoring fraction (e.g., bottom 25%) in terms of $S^{\text{final}}_{\ell,i}$, optionally restricted to a whitelist of layers.
   d) **Benefit stage:** for each shortlisted filter, compute a compute-aware benefit $B_{\ell,i}$ estimating MAC savings if the filter is removed, combining MAC reductions in the current convolutional layer, the next convolutional layer, and the first linear layer (where applicable) using cached shapes and MAC formulas, and enforce head coupling by maintaining a minimum input dimension for the first linear layer.
   e) **Selection and chunking:** given the Global budget for round r, sort all shortlisted filters by increasing $S^{\text{final}}_{\ell,i}$ and, for ties, by decreasing benefit $B_{\ell,i}$; traverse this ordering and select filters greedily while respecting the per-round budget and per-layer floors enforced by `ChannelTracker`, then partition the selected filters into chunks of size `CHUNK_SIZE` (e.g., 2–4) and record the intended picks in the reporter.
   f) **Guarded pruning for each chunk:**
      - Create a `RollbackSnapshot` (state dict, channel counts, trajectory length, (r, k)).
      - Apply structural surgery for all filters in the chunk (prune convolution and BN parameters, resize `ChannelGate` masks and affected downstream layers, update `ChannelTracker` and `PruneReporter`).
      - Optionally perform BN recalibration on $(X_{\text{bn}}, y_{\text{bn}})$ after a configured number of accepted chunks.
      - Run a fast evaluation on $(X_{\text{active}}, y_{\text{active}})$ with `FWD_EVAL` to obtain new accuracies A, A_c; update step-wise and cumulative drops.
      - If any step-wise or cumulative drop in overall or per-class accuracy exceeds the Global guard-rail limits for this dataset (Section III), rollback from the snapshot, resynchronize channels and `ChannelGate` masks, and soft-lock the associated layers/filters; otherwise accept the chunk, update last accepted accuracies and budgets, and append to the trajectory.
   g) At the end of round r, log per-layer before/after channel counts and cumulative picks, and optionally adjust chunk size or round pace for the next round.
6) After all rounds, perform a final BN recalibration on $(X_{\text{bn}}, y_{\text{bn}})$, run a full evaluation on L_{eval} to obtain final metrics, and compute Global compression statistics, layer-wise channel changes, timing breakdowns, and latency speedups for a structured CSV summary.

V. Experimental Setup

In this section, we describe the training procedure used to obtain baseline models, the pruning protocols for the Local HC and Global approaches, the evaluation metrics, and the hardware environment.

A. Training Protocol and Baselines

For each combination of architecture and dataset we first train a baseline model from scratch and then keep its weights fixed during pruning. All reported accuracy drops are measured relative to these baselines.

We train using standard stochastic gradient descent with momentum and weight decay, following common practice for CNNs and VGG-style networks. For both datasets we use mini-batches of size 64 for training and 512 for evaluation, as reflected in the experiment configuration. Training continues until validation accuracy on the evaluation split plateaus, at which point we save the model checkpoint as the pruning baseline.

On CelebA we train both the 5-layer CNN and VGG16 on the 10k training split, using the 19,867-image evaluation split for validation and later for guard-rail checks. On CIFAR10 we similarly train both architectures on 10k training images and treat the 2k evaluation split as both validation and pruning-time evaluation set. We do not perform any fine-tuning after pruning; all post-pruning accuracies are measured without additional retraining.

B. Pruning Runs and Canonical Experiments

We define a set of *canonical* pruning experiments that form the basis of our comparison. These experiments cover both approaches, both architectures, and both datasets:

- Local HC + 5-CNN + CelebA (floors S2),
- Local HC + VGG16 + CelebA (floors S2),
- Global + 5-CNN + CelebA (floors S2),
- Global + VGG16 + CelebA (floors S2),
- Global + 5-CNN + CIFAR10 (floors S2, relaxed guard rails),
- Global + VGG16 + CIFAR10 (floors S2, relaxed guard rails).

Each experiment is run for up to 75 pruning rounds, with micro-chunk sizes in the range of 2–4 filters per proposal for the Global method and individual cluster merges in the Local HC method. For each canonical run we record detailed JSON and CSV summaries containing baseline and final metrics, layer-wise compression, timing information, and guard-rail parameters. These summaries are then aggregated into a single master summary CSV (denoted `experiment_summary.csv` in our code), which we use to generate the tables and figures in Section VI.

We also conduct additional exploratory runs with alternative floor schemes, including more conservative floors such as S5 and more aggressive variants than S2, to study the impact of floors on compression and stability.

C. Metrics

We report the following metrics for each experiment:

- **Overall accuracy** before and after pruning, measured on the full evaluation split for the dataset.
- **Per-class accuracy** for each class, before and after pruning, and the corresponding drops in percentage points (pp).

- **Parameter counts** before and after pruning, including both convolutional and fully connected layers, and the total percentage of parameters pruned.
- **Convolutional MACs per image**, estimated per forward pass by summing the multiply-accumulate operations across all convolutional layers, and the ratio of MACs after vs. before pruning.
- **Latency per image**, measured in milliseconds per sample by timing batched forward passes on the GPU at evaluation batch size, and the resulting speedup factor.
- **Pruning overhead**, captured by the total wall-clock time of the pruning run and, for some analyses, decomposed into scoring, benefit evaluation, and post-pruning evaluation cost.
- **Guard-rail budget usage**, which summarizes how much of the allowed accuracy budget each run actually spends. Let $\Delta_{\max}^{\text{overall}}(D)$ and $\Delta_{\max}^{\text{class}}(D)$ denote the dataset-specific limits on overall and per-class drops, and let A_{final} and $A_{c,\text{final}}$ denote final accuracies. We define

$$B_{\text{overall}} = \frac{A_{\text{base}} - A_{\text{final}}}{\Delta_{\max}^{\text{overall}}(D)}, \tag{3}$$

$$B_{\text{class}} = \max_{c \in \{0,1\}} \frac{A_{c,\text{base}} - A_{c,\text{final}}}{\Delta_{\max}^{\text{class}}(D)}. \tag{4}$$

  For CelebA we have $\Delta_{\max}^{\text{overall}} = 2$ pp and $\Delta_{\max}^{\text{class}} = 6$ pp, and for CIFAR10 we use 3 pp and 9 pp respectively.
- **Economic return on investment (ROI)** of pruning, based on how quickly the one-time pruning overhead is amortized by faster inference. Let $T_{\text{eval}}^{\text{before}}$ and $T_{\text{eval}}^{\text{after}}$ denote the wall-clock time to evaluate the full validation split once before and after pruning, and let T_{opt} denote the total optimization overhead for the pruning run. The time saved per evaluation pass is

$$\Delta T_{\text{eval}} = T_{\text{eval}}^{\text{before}} - T_{\text{eval}}^{\text{after}}, \tag{5}$$

  and the break-even number of evaluation passes N_{BE} and images I_{BE} are

$$N_{\text{BE}} = \frac{T_{\text{opt}}}{\Delta T_{\text{eval}}}, \tag{6}$$

$$I_{\text{BE}} = N_{\text{BE}} \cdot N_{\text{eval}}, \tag{7}$$

  where N_{eval} is the size of the evaluation split (19,867 for CelebA, 2,000 for CIFAR10). For all canonical runs we instantiate these quantities using the measured evaluation times and optimization overheads from our master summary CSV, and report the resulting break-even points in Table V. As discussed in Section VI, Local HC runs incur non-trivial overhead but yield smaller speedups than the Global runs, leading to larger break-even counts.

All metrics are computed using the same evaluation pipeline for both approaches to ensure comparability. Where applicable we report mean values over multiple evaluation passes; however, because each canonical run is deterministic given its configuration, we primarily report single-run results.

D. Hardware and Implementation Details

All experiments are run on Google Colab using NVIDIA A100 GPUs with 40 GB of device memory. For some Global VGG16 runs we use the high-RAM runtime option to avoid out-of-memory errors during the most expensive scoring phases. Data loading uses PyTorch `DataLoader` with two worker processes and `pin_memory` enabled.

To ensure reproducibility, all canonical runs use a fixed random seed (42) for Python, NumPy, and PyTorch, including the generators used for `DataLoader` workers and for any stratified sampling of data frames or indices. This keeps training, evaluation, and pruning trajectories deterministic given the configuration.

Our implementation is based on PyTorch, with optional use of a graph-compilation step provided by the framework to fuse operations and reduce Python overhead when the architecture is stable (for example, during repeated evaluation of a fixed pruned model). To prevent issues related to compilation and dynamic module changes, we only enable this compiled-evaluation mode after the pruning layout has settled and avoid switching back and forth between compiled and eager modes without re-instantiation. These implementation choices are necessary to keep long multi-round pruning runs (up to 75 rounds) robust on real hardware. The complete pruning codebase, including Local HC and Global approaches and summary-generation utilities, is organized such that changing the architecture or dataset requires only a small number of configuration toggles.

For reproducibility, the main pruning thresholds and active-set sizes used in our canonical runs are exposed as configuration fields in the codebase. On CelebA we set the overall and per-class budgets to 2 pp and 6 pp, with per-chunk limits of approximately 1 pp overall and 3 pp per class and cumulative drops capped at these budgets; on CIFAR10 we use 3 pp and 9 pp. The active scoring subset $\mathcal{D}_{\text{score}}$ is capped at a few thousand images (for example, by setting an `ACTIVE_MAX_SAMPLES` parameter to around 3,000) and is constructed by a class-balanced, hardness-aware sampler as described in Section III. The ILR module uses default weights $(w_A, w_G, w_H) = (0.6, 0.4, 0.2)$ for activation RMS, BN-γ, and HRank signals, and the Global scorer uses $(\alpha_T, \alpha_H, \alpha_G) = (0.5, 0.3, 0.2)$ for the Taylor/HRank/BN-γ fused score when ILR is disabled in ablation mode. These configuration values match the defaults in our released scripts.

VI. Results

In this section we present the empirical results for the six canonical pruning experiments. We first summarize the overall accuracy and compression trade-offs across all runs, then quantify guard-rail budget usage and economic ROI, and finally delve into architecture- and dataset-specific observations.

A. Overall Comparison Across Six Experiments

Table I reports baseline and final accuracies, per-class drops, parameter reduction, convolutional MAC ratios, and latency speedups for all six canonical runs. In all cases the guard rails

are respected: on CelebA the overall accuracy drop stays below 2 pp and the maximum per-class drop below 6 pp; on CIFAR10 the overall drop remains below 3 pp and the per-class drops below 9 pp.

Overall, the Global scoring approach consistently achieves higher compression and larger speedups than the Local HC approach under the same guard rails. On the 5-CNN with CelebA, Global pruning removes approximately 70% of parameters and reduces convolutional MACs to about 61% of the baseline, yielding a $1.28\times$ latency speedup, while the Local HC method removes roughly 46% of parameters, reduces convolutional MACs to about 85% of the baseline, and achieves a more modest $1.04\times$ speedup. On VGG16 with CelebA, Global pruning removes about 46% of parameters compared to about 30% for Local HC, with convolutional MAC ratios of 0.72 vs. 0.75 (28% vs. 25% reduction relative to baseline) and latency speedups of $1.21\times$ vs. $1.03\times$ (21% vs. 3% faster).

On CIFAR10, where only the Global method is evaluated, we observe that even relatively modest parameter reductions (about 8% for 5-CNN and 5% for VGG16) can translate into substantial latency gains when pruning is focused on compute-intensive layers: the 5-CNN architecture achieves a $2.05\times$ speedup with a convolutional MAC ratio of 0.70, while VGG16 achieves a $1.52\times$ speedup with a MAC ratio of 0.68, all within the relaxed guard rails.

To briefly comment on each result in Table I:

- *Local HC, 5-CNN, CelebA:* achieves moderate compression (46% parameters pruned, MAC ratio 0.85) and a small latency gain (4%) while staying comfortably inside both overall and per-class guard rails.
- *Local HC, VGG16, CelebA:* prunes about 30% of parameters with a MAC ratio of 0.75 and a 3% speedup, indicating some redundancy can be removed without significantly disturbing per-class accuracy.
- *Global, 5-CNN, CelebA:* aggressively prunes 70% of parameters and reduces MACs to 61% of baseline for a 28% speedup, nearly saturating the overall accuracy budget but remaining within per-class limits.
- *Global, VGG16, CelebA:* prunes 46% of parameters and reduces MACs to 72% of baseline, yielding a 21% speedup with an overall drop of just under 2 pp and relatively small per-class degradation.
- *Global, 5-CNN, CIFAR10:* removes only about 8% of parameters yet delivers the largest speedup ($2.05\times$), reflecting that pruning is concentrated in compute-intensive layers.
- *Global, VGG16, CIFAR10:* provides a $1.52\times$ latency speedup with a MAC ratio of 0.68 and a 2.6 pp overall accuracy drop, again within the relaxed CIFAR10 guard rails.

In terms of compression levels, our Global VGG16 run on CelebA (45.6% parameters pruned, MAC ratio 0.72, ≈ 2 pp overall accuracy drop) falls within the broad range reported for structured pruning on VGG-style networks, but at the lower end of the parameter-pruning spectrum (many methods report 60–95% parameter reduction). For convenience, we label the six canonical experiments from Table I as E1–E6. Table II summarizes representative results from L1 filter pruning [1], Network Slimming [6], HRank [5], EPruner [15], and Deep Fisher pruning [16] on VGG16/VGG-style models, and juxtaposes them with our experimental setups (E1–E6). Broadly, prior work reports pruning roughly 60–95% of parameters and 34–80% of FLOPs with at most 1–3 pp accuracy loss, but without explicit per-class guard rails; our runs achieve comparable MAC ratios and accuracy drops while operating under stricter overall and per-class constraints and, in the case of the 5-CNN, on a smaller architecture where we are not aware of directly comparable structured-pruning baselines.

From Table II and Table I together, we see that our Global VGG16 runs (E4 on CelebA and E6 on CIFAR10) *operate in broadly similar compression regimes* to established structured-pruning methods on VGG-style networks, but generally at the more conservative end of the spectrum. On CelebA, E4 prunes about 46% of parameters and reduces convolutional MACs to 72% of the baseline with an overall drop of roughly 2 pp under explicit per-class guard rails, whereas Deep Fisher pruning on CelebA reports $> 95\%$ parameter pruning and a MAC/FLOP ratio of ~ 0.20 with no reported accuracy loss [16]. On CIFAR10, methods such as L1 filter pruning, Network Slimming, HRank, and EPruner typically prune 60–90% of parameters with MAC/FLOP ratios in the 0.20–0.66 range and accuracy drops between 0 and 3 pp [1], [6], [5], [15], whereas our Global VGG16 CIFAR10 run (E6; 5.2% parameters pruned, MAC ratio 0.68, overall drop 2.6 pp) is intentionally more conservative, reflecting tighter per-class guard rails and a design choice to avoid aggressive compression in this first study.

For the custom 5-CNN experiments (E1, E3, E5) we do not find directly comparable structured-pruning baselines in the literature: most published work targets larger architectures on CIFAR10 or CelebA. In this regime our results therefore serve more as reference points than as head-to-head comparisons, showing that even relatively small architectures can sustain substantial structured pruning (up to 70% parameters and MAC ratio 0.61 on CelebA in E3) under explicit guard rails.

Overall, we view the reported compression levels as indicative operating points under a fixed guard-rail and floor configuration, rather than as attempts to exhaust the space of possible optimizations. More aggressive combinations of floor schemes, scoring variants, and post-pruning fine-tuning—especially when coupled with relaxed per-step and per-class constraints—are likely to move Global runs closer to the most compressed regimes in Table II, but exploring such configurations lies beyond the scope of this first comparative study.

Figure 1 shows baseline vs. final overall accuracy for each run, while Fig. 2 highlights the per-class accuracy changes across all six experiments. Figures 3 and 4 plot the relationship between parameter pruning, MAC ratios, and latency speedup.

B. Guard Rail Budget Usage

Table III summarizes the fraction of the available accuracy budget used by each canonical run, in terms of both overall

TABLE I

SUMMARY OF CANONICAL PRUNING EXPERIMENTS. FOR EACH APPROACH, ARCHITECTURE, AND DATASET WE REPORT BASELINE AND POST-PRUNING ACCURACIES, OVERALL AND WORST PER-CLASS ACCURACY DROPS, PARAMETER COMPRESSION, CONVOLUTIONAL MAC RATIO, AND END-TO-END LATENCY SPEEDUP. ACCURACY DROPS ARE COMPUTED FROM FULL-PRECISION ACCURACIES IN THE EXPERIMENT SUMMARY AND MAY DIFFER SLIGHTLY FROM THE DIFFERENCE OF THE ROUNDED VALUES SHOWN.

Approach	Architecture	Dataset	Acc. before [%]	Acc. after [%]	Δ overall [pp]	Δ max class [pp]	Params pruned [%]	Conv MAC ratio	Latency speedup
Local HC	5-CNN	CelebA	94.15	92.41	1.74	5.43	46.03	0.85	1.04
Local HC	VGG16	CelebA	97.93	96.45	1.48	2.43	30.49	0.75	1.03
Global	5-CNN	CelebA	94.15	92.15	1.99	4.20	70.11	0.61	1.28
Global	VGG16	CelebA	97.93	95.94	1.99	2.15	45.62	0.72	1.21
Global	5-CNN	CIFAR10	91.45	90.25	1.20	2.70	7.89	0.70	2.05
Global	VGG16	CIFAR10	97.95	95.35	2.60	2.80	5.16	0.68	1.52

TABLE II

COMPARISON OF OUR SIX CANONICAL EXPERIMENTS (E1–E6, AS DEFINED IN TABLE I) WITH REPRESENTATIVE STRUCTURED PRUNING METHODS ON VGG-STYLE CNNS. LITERATURE NUMBERS ARE TAKEN FROM THE CITED PAPERS AND CORRESPOND TO VGG16(-BN) ON CIFAR10 OR CELEBA WITHOUT EXPLICIT PER-CLASS GUARD RAILS; MAC RATIOS ARE APPROXIMATED FROM REPORTED FLOP REDUCTIONS.

Exp	**Our setup**	**Representative literature setup**	**Lit. params pruned (%)**	**Lit. MAC/ FLOP ratio**	**Lit. ΔAcc (pp)**	**Notes**
E1	Local 5-CNN, CelebA	*No directly comparable small-CNN CelebA pruning result*	–	–	–	Custom 5-layer CNN; most prior work targets deeper architectures (e.g., VGG, ResNet).
E2	Local VGG16, CelebA	Li et al. [1], VGG16-BN on CIFAR10 (moderate compression)	≈ 64	~ 0.66	≈ 0	∼34% FLOP reduction with negligible accuracy change.
E3	Global 5-CNN, CelebA	*No directly comparable small-CNN CelebA pruning result*	–	–	–	We treat our Global 5-CNN run as a reference point for this architecture/dataset pair.
E4	Global VGG16, CelebA	Tian et al. [16], VGG-style CNN on CelebA	> 95	~ 0.20	≈ 0	Deep Fisher pruning; about 80% FLOPs removed without reported accuracy loss.
E5	Global 5-CNN, CIFAR10	*No small-CNN structured-pruning baseline identified*	–	–	–	Structured pruning on CIFAR10 typically targets VGG or ResNet; small 5-layer CNNs are rarely reported.
E6	Global VGG16, CIFAR10	Li et al. [1], Liu et al. [6], Lin et al. [5], Lin et al. [15]	60–90	0.20–0.66	0–3	Representative range over L1, Network Slimming, HRank, and EPruner on VGG16(-BN) for CIFAR10.

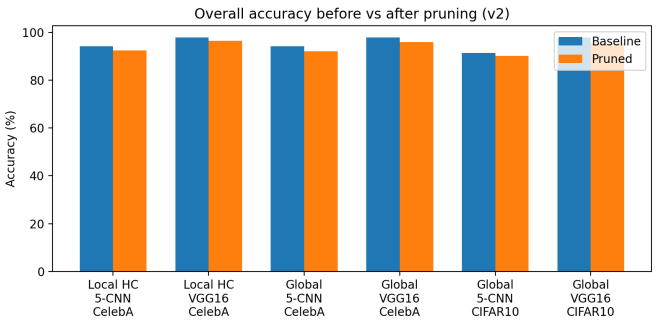

Fig. 1. Overall accuracy before vs. after pruning for the six canonical experiments. All runs satisfy their respective accuracy guard rails.

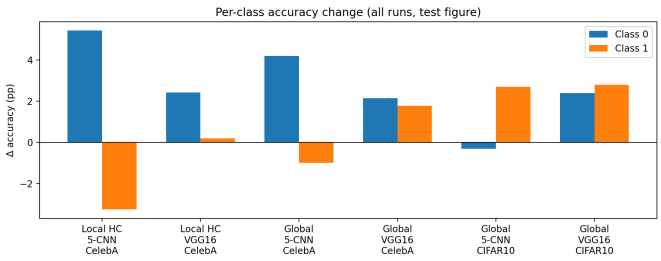

Fig. 2. Per-class accuracy changes for the six canonical experiments. Bars show the change in accuracy (in percentage points, pp) relative to baseline; positive values indicate an accuracy drop (worse), negative values indicate an accuracy gain (better).

accuracy and the worst per-class accuracy. Values are expressed as percentages of the dataset-specific budgets.

On CelebA, the Global runs almost fully exhaust the 2 pp overall budget ($\approx$ 99.4–99.7%), while the Local HC runs consume only about 74–87% of that budget. For per-class accuracy, the Local HC 5-CNN run uses about 91% of the 6 pp per-class budget—it pushes the harder class close to the limit—whereas the Global 5-CNN uses about 70% of the per-class budget and the VGG16 runs use only 36–41%. On CIFAR10, both Global runs stay within the relaxed budgets, using only 30–31% of the per-class budget and 40–87% of the overall budget, indicating headroom for more aggressive pruning, for example by lowering per-block floors (e.g., switching from the S2 to the more aggressive S1 floor configuration), increasing the per-round pruning budget, or modestly relaxing step-wise guard-rail thresholds.

C. CelebA: 5-CNN Architecture

On the 5-CNN with CelebA, the contrast between Local HC and Global pruning is most pronounced. With the shared S2 floors, the Local HC method reduces total parameters by about 46% and convolutional MACs by roughly 15% (MAC ratio 0.85), resulting in a $1.04\times$ latency speedup. Overall accuracy drops by 1.74 pp (about 87% of the allowed overall budget), and per-class drops remain within the 6 pp limit. Notably, class 0 (the majority class) experiences a drop of approximately 5.4 pp, while class 1 sees an improvement of about 3.3 pp, indicating a rebalancing effect that favors the harder class and uses around 91% of the per-class budget.

The Global scoring method, under the same S2 floors, is significantly more aggressive. It removes roughly 70% of

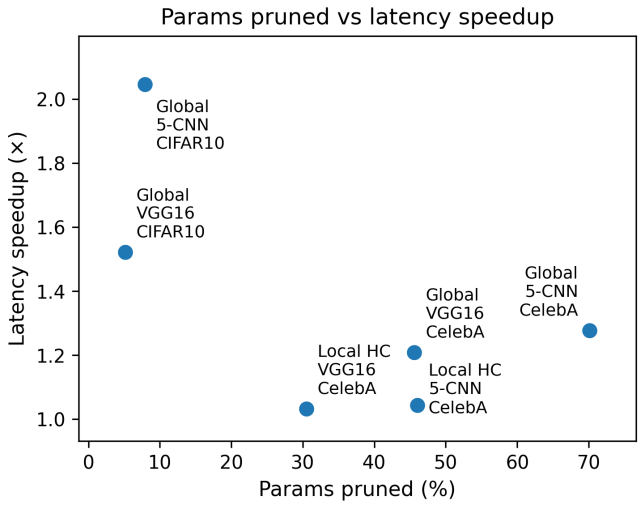

Fig. 3. Parameter pruning vs. latency speedup across all six canonical experiments. Global pruning on the 5-CNN (CelebA) achieves the highest parameter compression, while Global 5-CNN on CIFAR10 exhibits the largest speedup despite modest parameter reduction.

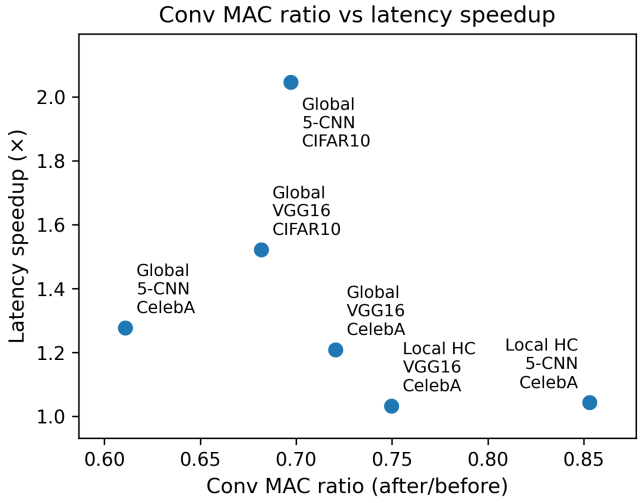

Fig. 4. Convolutional MAC ratio vs. latency speedup. The chart illustrates that targeting compute-intensive layers can yield substantial speedups even when overall parameter reduction is limited. For the Global 5-CNN CIFAR10 run, the strong speedup arises from pruning being concentrated in compute-intensive intermediate blocks (especially the fourth block), which dominate the MAC budget, while leaving the earliest blocks largely unchanged.

parameters and reduces convolutional MACs to about 61% of the baseline, yielding a $1.28\times$ latency speedup. The overall accuracy drop is 1.99 pp, essentially saturating the 2 pp budget ($\sim 99.7\%$ of the allowed drop), while per-class drops (about 4.2 pp for class 0 and a 1 pp improvement for class 1) use about 70% of the per-class budget. This behavior illustrates that the Global method tends to spend the overall budget fully, trading margin on the easier class for deeper compression and compute savings, but without violating guard rails.

TABLE III
GUARD RAIL BUDGET USAGE FOR EACH CANONICAL EXPERIMENT. "OVERALL BUDGET USED" AND "CLASS BUDGET USED" ARE PERCENTAGES OF THE ALLOWED DROPS (2 PP OVERALL AND 6 PP PER CLASS FOR CELEBA; 3 PP AND 9 PP FOR CIFAR10). PERCENTAGES ARE COMPUTED FROM FULL-PRECISION ACCURACIES IN THE EXPERIMENT SUMMARY, NOT DIRECTLY FROM THE ROUNDED DROPS SHOWN.

Experiment	Overall drop [pp]	Overall budget used [%]	Max class drop [pp]	Class budget used [%]
Local HC 5-CNN CelebA	1.74	86.8	5.43	90.6
Local HC VGG16 CelebA	1.48	74.0	2.43	40.5
Global 5-CNN CelebA	1.99	99.7	4.20	70.0
Global VGG16 CelebA	1.99	99.4	2.15	35.8
Global 5-CNN CIFAR10	1.20	40.0	2.70	30.0
Global VGG16 CIFAR10	2.60	86.7	2.80	31.1

D. CelebA: VGG16 Architecture

On VGG16 with CelebA, both approaches behave more conservatively. With the same S2 floors, the Local HC method removes about 30% of parameters and reduces convolutional MACs to approximately 75% of the baseline (MAC ratio 0.75), achieving a $1.03\times$ latency speedup. Overall accuracy drops by 1.48 pp (about 74% of the overall budget), and per-class drops are small (around 2.4 pp for class 0 and 0.2 pp for class 1), using roughly 40% of the per-class budget and suggesting that the method prunes away some redundancy while largely preserving the classifier's balance.

The Global method, again with S2 floors, removes about 46% of parameters and reduces convolutional MACs to roughly 72% of the baseline (MAC ratio 0.72), yielding a $1.21\times$ speedup. The overall accuracy drop is again close to 2 pp (about 99.4% of the budget), and per-class drops are around 2.1 pp and 1.8 pp for the two classes (36% of the per-class budget). Compared to the small CNN, VGG16 still appears less over-parameterized under these guard rails: even with more aggressive pruning, Global cannot match the $\sim 70\%$ parameter reduction achieved on the 5-CNN without saturating the accuracy budget. Nonetheless, Global pruning offers a clear advantage over Local HC pruning in terms of compute savings.

E. CIFAR10: 5-CNN and VGG16

For CIFAR10 we evaluate only the Global method, using the same S2 floor scheme as on CelebA but relaxed guard rails (± 3 pp overall, ± 9 pp per class) due to the smaller evaluation set. Interestingly, the Global method does not actually saturate these relaxed limits. On the 5-CNN architecture the overall accuracy drop is about 1.2 pp (40% of the overall budget), with per-class changes of roughly -0.3 pp for one class and $+2.7$ pp for the other (30% of the per-class budget). Parameter reduction is modest (about 7.9%), but convolutional MACs drop to 70% of the baseline and latency improves by $2.05\times$. Inspecting the per-layer pruning logs (filter counts before vs. after pruning, not shown here for brevity) shows that, compared to the CelebA run, the CIFAR10 5-CNN run shifts more of its pruning to intermediate convolutional blocks, notably the third and especially the fourth block, while leaving the very first block unchanged; these intermediate blocks account for a large share of the convolutional MACs. This concentration

TABLE IV
PER-BLOCK CHANNEL COUNTS FOR THE 5-CNN UNDER GLOBAL PRUNING ON CELEBA AND CIFAR10. VALUES ARE CHANNELS PER CONVOLUTIONAL BLOCK BEFORE AND AFTER PRUNING, WITH THE PERCENTAGE OF CHANNELS PRUNED RELATIVE TO BASELINE.

Block	CelebA			CIFAR10		
	Before	After	Pruned [%]	Before	After	Pruned [%]
B1 (features.0)	64	64	0.0	64	64	0.0
B2 (features.4)	128	116	9.4	128	128	0.0
B3 (features.8)	256	256	0.0	256	232	9.4
B4 (features.12)	512	128	75.0	512	184	64.1
B5 (features.16)	512	160	68.8	512	512	0.0

of pruning in compute-intensive intermediate blocks highlights that modest overall parameter reduction can still yield a large end-to-end speedup when pruning targets the right parts of the network. Table IV summarizes the per-block channel counts for the 5-CNN Global runs on CelebA and CIFAR10.

On VGG16 the overall accuracy drop is about 2.6 pp (about 87% of the budget) and per-class drops are around 2.4–2.8 pp (31% of the per-class budget), still within the relaxed guard rails. Parameter reduction is about 5.2%, but convolutional MACs fall to roughly 68% of the baseline and latency improves by $1.52\times$. Again, targeting compute-intensive layers yields non-trivial speedups even when the total parameter count is only slightly reduced. These results suggest that, at least for CIFAR10, the current floors and chunk budgets are more conservative than necessary: the method stops pruning before exhausting the relaxed per-class budget, leaving room for future improvements. Using a more aggressive floor configuration such as S1 (with lower per-block floors) would likely have yielded additional compression and MAC reductions.

F. Economic ROI: Optimization Overhead and Break-even Usage

We now examine the economic ROI of pruning by comparing one-time optimization overhead to the per-pass latency savings. Table V reports the pruning overhead, latency speedup, and break-even points for all six canonical runs. For the Global runs, the combination of multi-hour overhead and substantial latency gains (up to $2.05\times$) yields break-even points between roughly 170 and 820 full evaluation passes, corresponding to 0.34–16.3 million evaluation-sized images. For the Local HC runs on CelebA, the overhead is still significant—about 1.10 hours for the 5-CNN and 2.83 hours for VGG16—but the associated speedups are much smaller ($1.04\times$ and $1.03\times$), leading to higher break-even points (roughly 1,700 and 5,600 passes, or about 33.9 and 110.4 million images, respectively). These numbers show that Global pruning is particularly attractive for high-throughput or long-lived deployments, where the one-time overhead can be amortized over many inference runs, while Local HC pruning, despite its simpler implementation, incurs comparable-order overhead but yields much smaller speedups, resulting in substantially larger break-even counts.

On CelebA, the Global 5-CNN run requires about 446 full evaluation passes (approximately 8.9 million evaluation-sized images) to amortize its ~ 2.3 hour pruning overhead, while the Global VGG16 run requires about 818 passes (roughly 16.3 million images). On CIFAR10, the break-even points are substantially lower in terms of images due to the much smaller evaluation set: roughly 169 passes (0.34 million images) for the Global 5-CNN and 658 passes (1.32 million images) for Global VGG16. These numbers show that Global pruning is particularly attractive for high-throughput or long-lived deployments, where the one-time overhead can be amortized over many inference runs, while Local HC pruning, despite its simpler implementation, incurs comparable-order pruning overhead but yields much smaller speedups, resulting in substantially larger break-even counts.

TABLE V
ECONOMIC ROI FOR CANONICAL PRUNING RUNS. "OVERHEAD" IS THE ONE-TIME PRUNING COST IN HOURS, "SPEEDUP" IS LATENCY SPEEDUP AFTER PRUNING, AND "BREAK-EVEN" IS THE NUMBER OF FULL EVALUATION PASSES AND EQUIVALENT NUMBER OF EVALUATION IMAGES REQUIRED TO AMORTIZE THE PRUNING OVERHEAD. N_{EVAL} IS 19,867 FOR CELEBA AND 2,000 FOR CIFAR10.

Experiment	Speedup	Overhead [h]	Break-even passes	Break-even images [M]
Local HC 5-CNN CelebA	1.04	1.10	1,706	33.90
Local HC VGG16 CelebA	1.03	2.83	5,559	110.45
Global 5-CNN CelebA	1.28	2.30	446	8.86
Global VGG16 CelebA	1.21	3.46	818	16.25
Global 5-CNN CIFAR10	2.05	0.81	169	0.34
Global VGG16 CIFAR10	1.52	2.52	658	1.32

VII. DISCUSSION

The experiments reveal several qualitative and quantitative differences between Local HC and Global pruning under accuracy guard rails.

A. Compression vs. Speed: MACs Matter More Than Parameters

Across all runs, parameter reduction, MAC reduction, and latency speedup do not align perfectly. For example, the Global 5-CNN on CelebA removes about 70% of parameters and reaches a convolutional MAC ratio of 0.61, yet only achieves a $1.28\times$ latency speedup, whereas on CIFAR10 it prunes about 8% of parameters with a MAC ratio of 0.70 but delivers a $2.05\times$ speedup. This shows that wall-clock latency depends not only on total parameters or MACs, but also on where computation happens (early vs. late layers), memory behavior, and library/kernel implementation. The Global method mitigates this by using benefit-aware selection that explicitly optimizes MAC savings and tends to focus on compute-intensive layers (often in early or intermediate convolutions); Local HC, which only reasons within layers and does not target MACs directly, spreads parameter cuts more uniformly and achieves smaller latency gains for the same accuracy budget. In practice, pruning methods should therefore be tuned primarily against measured latency, with parameter counts used as a secondary proxy, highlighting that practical speedups depend on implementation details in addition to sparsity [8], [4].

B. Guard Rails and Per-Class Behavior

Both methods respect the specified guard rails, but per-class results show that guard rails alone do not guarantee fairness across classes. On CelebA with the 5-CNN architecture, Local HC pruning reduces class 0 accuracy by about 5.4 pp while improving class 1 by about 3.3 pp, whereas Global pruning produces a smaller drop for class 0 and a smaller improvement for class 1; on CIFAR10, per-class changes are also asymmetric, though still within the relaxed limits. The budget-usage metrics in Table III make this explicit: the guard rails primarily enforce *stability* (caps on per-class drops), not fairness in the sense of preserving relative performance between classes. Practitioners who care about fairness may need additional objectives or constraints, for example penalizing changes in the gap between per-class accuracies or explicitly optimizing for balanced per-class performance alongside the existing guard rails.

C. Architecture Dependence and Redundancy

The amount of pruning possible under fixed guard rails depends strongly on the architecture. On CelebA, the custom 5-layer CNN tolerates aggressive pruning: Global scoring with S2 floors can remove roughly 70% of parameters while staying within a 2 pp overall accuracy drop, whereas VGG16 on the same dataset is less redundant in this regime and Global pruning reaches about 46% parameter reduction before saturating the accuracy budget. This suggests that Global pruning under guard rails can be used as a diagnostic for over-parameterization: large gaps between Local HC and Global compression on a given architecture (as for the 5-CNN, where Global reaches $\sim 70\%$ vs. $\sim 46\%$ for Local HC) indicate substantial untapped redundancy, while smaller gaps (as for VGG16, where Global reaches $\sim 46\%$ vs. $\sim 30\%$ for Local HC) indicate that the baseline model is closer to the minimum capacity required at the specified accuracy. The budget-usage metrics reinforce this view: on CelebA, both 5-CNN runs use a large fraction of the overall budget (about 87% and 100%), but the Global run converts that budget into much larger MAC and latency reductions than Local HC.

D. Local vs. Global: Behavioral Characteristics

From an algorithmic perspective, Local HC pruning and Global scoring exhibit distinct behavioral characteristics. Local HC tends to under-use the available overall accuracy budget (about 74–87% on CelebA) and, on the 5-CNN architecture, comes close to exhausting the per-class budget for the hardest class (around 91%); it stops when no more safe merges are available inside blocks, often before any guard rail is fully saturated in the VGG16 case. This makes Local HC a safe, predictable baseline, but leaves compression headroom unused. The Global method, by contrast, behaves like a budget-aware optimizer: on CelebA it deliberately spends roughly 99–100% of the allowed overall budget while using only 36–70% of the per-class budget, then relies on floors and soft locks to prevent further degradation; on CIFAR10 it still stays comfortably within the relaxed overall and per-class limits, reflecting the smaller evaluation set and the chosen S2 floor scheme. The price of this efficiency is higher algorithmic and engineering complexity: Global pruning requires careful handling of multi-signal scoring, compute-aware benefit estimation, chunk scheduling, caching, rollback logic, and economic ROI (Table V). Where long-lived or high-throughput deployments justify this complexity, Global pruning offers clear advantages; in smaller or simpler deployments, Local HC remains an attractive and easier-to-maintain choice.

VIII. Limitations and Future Work

Our study has several limitations. First, we focus on binary classification tasks derived from CelebA and CIFAR10. These tasks are useful for highlighting per-class effects, but they do not cover multi-class settings where per-class guard rails would need to scale to more classes and potentially more complex fairness notions.

Second, the CIFAR10 evaluation set is relatively small (around 2,000 images), which increases the variance of accuracy estimates and motivated our use of relaxed guard rails. While the Global method remains within these limits, the small evaluation size makes it harder to draw fine-grained conclusions about per-class behavior. Future work should incorporate larger and more diverse evaluation sets to stress-test the guard-rail mechanism.

Third, we consider only two architectures: a custom 5-layer CNN and VGG16. Modern deployments often use architectures such as ResNets, MobileNets, or vision transformers. Extending both pruning approaches and guard-rail logic to such models is a natural next step.

Fourth, we do not explore post-pruning fine-tuning. Fine-tuning is known to partially recover accuracy after pruning and could interact with guard rails in interesting ways, for example by allowing more aggressive pruning followed by supervised recovery. Studying how Local HC and Global pruning behave when combined with structured fine-tuning under accuracy constraints is left for future work.

Finally, our ROI analysis focuses on evaluation-time speedups. In real deployments, request patterns, batch sizes, and hardware utilization further influence the true economic benefit of pruning. Extending the ROI model to production traces and energy consumption would provide a more complete picture.

IX. Conclusion

We presented a comparative study of two structured pruning paradigms for convolutional networks under explicit accuracy guard rails. A Local hierarchical clustering (Local HC) method provides a simple, conservative baseline that yields moderate compression and sometimes beneficial rebalancing of classes, while a Global scoring method with compute-aware selection and strong engineering support can exploit the full accuracy budget to deliver substantially higher parameter and MAC reductions and larger latency gains. By introducing derived metrics for guard-rail budget usage and economic ROI, we connect pruning decisions more directly to deployment constraints.

Our results on CelebA and CIFAR10, across a small 5-CNN and VGG16, show that both paradigms can respect strict constraints on overall and per-class accuracy, but with very different compression profiles and economic characteristics. The absolute pruning levels we report should therefore be interpreted primarily as indicative operating points under a particular choice of floors, guard rails, and optimization budget, not as attempts to match the most aggressive compression results in the literature. Within this regime, the Global method demonstrates greater flexibility and control—it can systematically trade accuracy margin for compression and speed—whereas Local HC retains advantages in implementation simplicity and shorter optimization time. We expect that the methodology and implementation developed in this work will be useful to practitioners who need to deploy pruned models under explicit accuracy and cost constraints, and that it will provide a foundation for future work on fairness-aware and deployment-aware pruning under guard rails.

REFERENCES

[1] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning filters for efficient convnets," in *International Conference on Learning Representations (ICLR)*, 2017.

[2] W. Wen, C. Wu, Y. Wang, Y. Chen, and H. Li, "Learning structured sparsity in deep neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 29, 2016, pp. 2074–2082.

[3] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient inference," in *International Conference on Learning Representations (ICLR)*, 2017.

[4] T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste, "Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks," *Journal of Machine Learning Research*, vol. 22, no. 241, pp. 1–124, 2021.

[5] M. Lin, R. Ji, Y. Wang, Y. Zhang, B. Zhang, Y. Tian, and L. Shao, "Hrank: Filter pruning using high-rank feature map," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 1529–1538.

[6] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2736–2744.

[7] J. Ye, X. Lu, Z. Lin, and J. Z. Wang, "Rethinking the smaller-norm-less-informative assumption in channel pruning of convolution layers," in *International Conference on Learning Representations (ICLR)*, 2018.

[8] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell, "Rethinking the value of network pruning," in *International Conference on Learning Representations (ICLR)*, 2019.

[9] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *International Conference on Learning Representations (ICLR)*, 2015.

[10] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015, pp. 448–456.

[11] Z. Liu, P. Luo, X. Wang, and X. Tang, "Deep learning face attributes in the wild," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 3730–3738.

[12] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.

[13] Y. LeCun, J. S. Denker, and S. A. Solla, "Optimal brain damage," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 2, 1990, pp. 598–605.

[14] B. Hassibi and D. G. Stork, "Second order derivatives for network pruning: Optimal brain surgeon," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 5, 1993, pp. 164–171.

[15] M. Lin, R. Ji, S. Li, Y. Wang, Y. Wu, F. Huang, and Q. Ye, "Network pruning using adaptive exemplar filters," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 12, pp. 7357–7366, 2021.

[16] K. Tian, L. Jing, Y. Mo, and Y. Liu, "Structured deep fisher pruning for efficient facial trait classification," *Image and Vision Computing*, vol. 77, pp. 45–59, 2018.